

CeTTa Development Roadmap

What Works, What’s Broken, What’s Planned

Zar Goertzel Oruži

2026-04-07 (living document)

Abstract

This document tracks the development state of CeTTa, a direct C runtime for the MeTTa language. It consolidates information from the published CeTTa paper, the conceptual architecture primer, 7 runtime specification files, 10 review bundles produced during development, Lean formalization artifacts, a full audit of the **core-opt** branch conducted 2026-04-07, and an independent GPT-5.4 Pro triage of the same branch. Its purpose is to prevent feature loss, regression blindness, and duplicated work across development sessions. It includes a *capability ledger* (§10) that tracks every nontrivial evaluator guarantee with its semantic witness, performance witness, and code anchor.

Contents

1	Current State: CeTTa Main (v1.3.2-dev)	2
1.1	Test Suite	2
1.2	Implemented Features	2
1.3	Lean Verification State	3
2	Performance Baseline	3
3	Core-Opt Epoch: Attempted Optimizations	4
3.1	Features Attempted	4
3.2	Collateral Damage	5
3.3	Codex “Fix” Session (Reverted)	5
3.4	GPT-5.4 Pro Triage (2026-04-07)	5
3.5	Verdict	6
4	Planned: Cherry-Pick Roadmap	6
5	Planned: Typed Ordinary Application	7
5.1	Problem	7
5.2	Solution Direction	7
5.3	Lean Oracle Support	8
6	Planned: Demand-Driven Evaluation	8
7	Planned: MORK and PathMap	8

8 Lean Formalization Frontier	9
8.1 Completed (sorry-free)	9
8.2 Needed	9
9 Specification Artifacts	9
10 Capability Ledger	10
11 Principles	11

1 Current State: CeTTa Main (v1.3.2-dev)

Branch `main`, commit `c3f5e67`.

1.1 Test Suite

Suite	Passed	Failed	Skipped
<code>make test</code> (oracle + profile)	340	0	50

The 50 skipped break down into: 17 MM2/MORK-specific, 2 pathmap regressions, 1 pretty-vars surface, 27 probe/translation files without `.expected` goldens, 3 Python-dependent tests in core builds.

1.2 Implemented Features

Arena allocation

64KB blocks, batch deallocation at scope exit. No per-atom `malloc/free`.

HashConsTable

Global term sharing on the persistent arena. Hash-consed atoms enable $O(1)$ structural equality. The eval arena does *not* have its own hashcons (see §3).

SymbolId dispatch

Symbol strings are interned into unique IDs at parse time. All dispatch is integer comparison, not `strcmp`. 38% instruction reduction vs. string-based dispatch.

Equation index

Equations (`= lhs rhs`) are hashed by head symbol into 256 buckets, with a `SubstTree` discrimination trie within each bucket. Epoch-tagged variables eliminate per-candidate re-naming.

Tail-call optimization

`TAIL_REENTER` trampoline (`goto tail_call`) for single-result `let`, `chain`, `let*`, `if`, `case`, `function/return`. 15 call sites in `eval.c`.

Structured space kinds

Queue (FIFO), hash ($O(1)$ lookup), stack (LIFO), in addition to the default equation-indexed space.

Pluggable match backends

Native hash-indexed, PathMap/MORK-backed, and compiled-space backends, selected per-space via `space-set-match-backend!`.

Two-stage stdlib

Standard library precompiled into a binary blob at build time (`cetta-stage0` $\rightarrow$ `-compile-stdlib` $\rightarrow$ blob $\rightarrow$ `cetta`). 6 ms startup.

Unlimited fuel

Default fuel is -1 (unlimited). Stack overflow caught via `sigaltstack` + SIGSEGV handler.

Effect classification

`cetta_effect_policy.h`: 13 effect classes with memo policy, scheduler policy, and lowering eligibility. Backed by Lean contracts.

Runtime counters

50 counters (hashcons inserts/hits, bindings operations, table hits/misses, space revision bumps, etc.) exposed via `runtime-stats!`.

Module system

`import!`, `include`, `register-module!`, `git-module!`, module inventory.

Python FFI

Default build links Python 3; `call-native!` dispatches to Python callables.

MORK bridge

Optional static or dynamic link to MORK/PathMap for persistent indexed spaces via `.act` format.

1.3 Lean Verification State

5,400+ lines of sorry-free Lean proof establishing:

- `HE` $\leftrightarrow$ `PeTTa` semantic correspondence on the pure fragment, including import commutativity.
- `EvalSpec` inductive operational semantics (`EvalAtom`, `MettaCall`, `InterpretExpression`, `InterpretArgs`, `InterpretFunction`, `InterpretTuple`).
- Typed-apply machine: 7 invariants, `ProvenanceInvariant`, `applyComplete_sound_structure`.
- `BindingConsistency`: loop-freedom preserved through evaluation.
- `OwnershipInvariant`: 3-way target safety classification for the running-stack contract.
- `TypedApplySoundness`: machine emissions $\rightarrow$ `InterpretArgs` derivations via `MachineArgChain`.

2 Performance Baseline

All measurements on the same machine, 2026-04-07.

Benchmark	PeTTa	CeTTa main	HE	Notes
countdown 100k	0.15 s / 17 MB	1.70 s / 347 MB	—	tail recursion
fib 25	0.23 s / 17 MB	5.48 s / 1023 MB	timeout	exponential
N-queens 8 [92]	—	2.69 s / 586 MB	timeout	nondeterministic
metamath d5 bc	31 s / 3.7 GB	OOM @ 6 GB	timeout	85K proof trees
backchain 60 ent	—	correct	correct	depth-1
matchjoin 6×6	—	correct (216)	correct	2-hop join

Key gap. PeTTa is 10–30× faster on recursive workloads. The advantage comes from the WAM (last-call optimization, heap-allocated environments, efficient backtracking) and SWI-Prolog’s tabling infrastructure. CeTTa’s `TAIL_REENTER` handles the single-result case but multi-result nondeterministic evaluation still allocates `OutcomeSet` arrays on every recursive call.

3 Core-Opt Epoch: Attempted Optimizations

Branch `core-opt`, commit 2b4b826 (“PROBABLY BUGGY WIP”). Test suite: 259 passed, 99 failed, 28 skipped. **95 extra failures** vs. main.

3.1 Features Attempted

Feature	File(s)	Status	Bug
Query cache with interleaved staging	<code>eval.c</code>	Buggy	<code>QueryCacheVisitCtx</code> interleaves staging and forwarding. Caution: GPT-5.4 Pro found that query caching is only active when <code>search-table-mode</code> is set to <code>variant</code> (not the default). The plain countdown $O(n^2)$ regression has a <i>different, unidentified</i> root cause (§3.4).
<code>bindings_clone_merge_grow</code> and <code>bindings_simple</code> fast path	<code>eval.c</code>	Buggy	Silent wrong merges. N-queens, unique, metamath return empty or wrong results.
Bindings lookup index (open-addressed hash)	<code>match.c</code>	Buggy	“Unsound in long-running workloads.” Incorrect lookups.
<code>match_pattern_candidates</code> + <code>needs_loop_check</code> skip	<code>space.c</code> , <code>space_match_backend.c</code>	Buggy	Skips <code>bindings_has_loop</code> when it shouldn’t. Produces looping bindings.
HashConsTable on eval arena (owned)	<code>main.c</code> , <code>atom.c</code>	Buggy	Invalidated on arena reset between <code>!</code> commands. <code>hashcons-insert</code> counter shows 0.

Feature	File(s)	Status	Bug
NativeResumeStack for let/chain	eval.c	Regression	Replaced <code>TAIL_REENTER</code> with full stack frame allocation. Destroys TCO that main already has. GPT-5.4 Pro confirmed this is a <i>semantic</i> regression, not merely a performance one: <code>countdown-let 5000</code> returns wrong answers, not just slowly. The <code>borrowed_hashcons</code> test failure was actually masked by this bug, not a real hash-cons issue.

3.2 Collateral Damage

In addition to the 6 buggy features, core-opt:

- Deleted `metta_eval_bind_typed` (type-directed evaluation used in 3 call sites on main).
- Deleted `cetta_effect_policy.h` (Lean-contract-backed effect classification).
- Deleted the 28-line canonicalization boundary comment in `term_universe.c` (with Lean verification references).
- Bloated `eval.c` from 7,320 to 12,596 lines (+72%) by adding 55 NativeResumeStack consumer/frame functions.
- Changed `bindings_clone` to `bindings_move` in `subst_tree.c` (potential use-after-free).

3.3 Codex “Fix” Session (Reverted)

A subsequent Codex session attempted to fix the bugs by *disabling all 6 optimizations*: hardcoding `return false` for the lookup index, removing the fast merge call, replacing `match_pattern_candidate` with `match_atoms`, and disabling the eval-arena hashcons entirely. This made tests pass (336/3/47) but produced a branch functionally identical to main — zero net-new work. The TCO for `let/chain` that Codex “re-added” already existed on main. These changes were reverted.

3.4 GPT-5.4 Pro Triage (2026-04-07)

An independent GPT-5.4 Pro review of the same bundle found that the bug ordering in our original diagnosis had drifted:

1. **The query-cache $O(n^2)$ diagnosis is stale.** Query caching is gated behind `search-table-mode = variant`, which is off by default. The plain `countdown 100k` regression has a *different, unidentified root cause* that persists even after restoring the `let/chain` fast path. After GPT-5.4 Pro’s surgical patch: `countdown 5K = 0.98s`, `10K = 5.82s` — still quadratic, but from a different subsystem.
2. **The `let/chain` regression is semantic.** `countdown-let 5000` returns assertion mismatches (wrong answers), not just slowness. The `borrowed_hashcons` test was a false red: it passed once `let/chain` were restored.
3. **One surgical patch fixes 6/6 bundled tests.** Restoring main’s eager-eval + `TAIL_REENTER` path for `let` and `chain` in `metta_call` moves the bundled fence from 3/6 to 6/6. Patch provided: `cetta_let_chain_tco_salvage.patch`.
4. **The `NativeResumeStack` work is not junk.** The mistake was applying the general stack machine where the cheap deterministic fast path already existed and was correct. The right design is hybrid: `TAIL_REENTER` for single-result fast paths, `NativeResumeStack` only for genuinely deferred/multi-result paths.
5. **Benchmark claims must be regenerated, not hand-carried.** If a root-cause story changes, the bug documentation must change with it. The current branch demonstrates exactly why.

3.5 Verdict

The core-opt branch is not mergeable. Treat it as a quarry of ideas and code, not a trustworthy head. The recommended path is to start a salvage branch from main and cherry-pick proven pieces one at a time (§4), with every step gated by the capability ledger (§10).

4 Planned: Cherry-Pick Roadmap

Start a salvage branch from main (340/0/50). Each step ports one piece, gated by the full capability ledger (§10).

- Step 1. Restore `let/chain` TCO + add canaries.** Apply GPT-5.4 Pro’s `cetta_let_chain_tco_salvage.patch`: restore main’s eager-eval + `TAIL_REENTER` path in `metta_call`. Add `countdown-let 5000` and `countdown-chain 5000` as permanent regression tests. Then *re-measure plain countdown* to identify the real root cause of the remaining quadratic scaling.
- Step 2. Diagnose the real countdown regression.** GPT-5.4 Pro confirmed the query cache is not the cause (it is off by default). Profile the actual hot path. Candidates: `OutcomeSet` reallocation, `bindings_clone` in equation dispatch, arena fragmentation, or a different quadratic interaction. Do not proceed to further optimizations until this is understood.
- Step 3. `TableStore` / `tabling`.** Self-contained (`table_store.c`, 209-line diff). Biggest potential win: variant tabling can memoize repeated subgoals in backward chaining, closing the metamath d5 gap with PeTTa. Requires the collect-then-visit query cache pattern (already on main).
- Step 4. HashCons ownership model.** Port `ARENA_HASHCONS_BORROWED` / `ARENA_HASHCONS_OWNED_LOCAL` from `atom.c/atom.h` (86-line diff). Fix the arena-reset invalidation bug *before* porting: the owned hashcons must survive `arena_reset` between `!` commands.

- Step 5. Bindings lookup index.** Port the open-addressed hash table from `match.c` (816-line diff). Fix the soundness bug first: write a C-level unit test that exercises large binding sets (≥ 64 entries) with collisions, then fix the index before enabling it.
- Step 6. `match_pattern_candidate`.** Port from `space.c` and `space_match_backend.c` (326 + 383 line diffs). Fix the `needs_loop_check` invariant: if the function says no loop check is needed, it must be *impossible* for the resulting bindings to contain a loop.
- Step 7. Query cache streaming** (if needed). Main’s collect-then-visit is correct and $O(n)$. Defer unless profiling shows it as a bottleneck.

Gate. Each step must satisfy:

- Full test suite: ≥ 340 passed, 0 failed.
- Benchmark matrix: no regression $> 10\%$ vs. main on any benchmark in §2.
- No feature deletions (no removing `cetta_effect_policy.h`, `metta_eval_bind_typed`, documentation, etc.).

5 Planned: Typed Ordinary Application

The remaining deep-benchmark blocker: typed ordinary-function application falls out of the native control flow into direct-style recursive C, causing stack overflow on `bench_bio_bc_deep.metta`.

5.1 Problem

When the evaluator encounters a typed application (`f arg1 arg2`) where `f` has a function type, it dispatches through `metta_call`, which re-enters evaluation recursively. For deep constructor-shaped typed terms (common in PLN backward chaining), this amplifies C stack usage until overflow.

5.2 Solution Direction

An explicit-stack typed argument evaluator:

1. Evaluate each argument with explicit accumulation (no C recursion).
2. Join barrier: wait for all arguments before call-term reconstruction.
3. Redispatch through the main evaluation loop (not `metta_call`).

This is the *right* place for the `NativeResumeStack` infrastructure — typed application is genuinely complex and needs frame-based control. But it must *not* replace the working `TAIL_REENTER` paths for `let/chain/if/case`.

5.3 Lean Oracle Support

From the Lean formalization side (per `CLAUDE_LEAN_SUPPORT.md`):

1. Formalize `IsValue` vs. `IsReducible` — make “no equations = value” formally impossible.
2. Formalize direct vs. frame-based typed application as two evaluators, prove they agree on outcomes.
3. Formalize the ownership invariant for deferred targets.
4. Formalize visible-binding projection / branch-delta correctness.

Existing Lean artifacts (`TypedApplyMachine.lean`, `OwnershipInvariant.lean`, `BindingConsistency.lean`) provide foundations; the gap is the `IsValue` split and the `direct≈frame` equivalence.

6 Planned: Demand-Driven Evaluation

The 4-phase roadmap from the April 5 development checkpoint:

Phase 0: Bug-first cleanup

`match.c` memo audit, conjunction lowering audit. Semantic fence only.

Phase 1: Conjunction ordering

Order conjuncts by selectivity estimate. Measure on medium ladder + variant bio.

Phase 2: Tabling cost reduction

Optimize the `TableStore` so tabling is a net win on medium and deep bio workloads. Measure variant vs. plain on medium + one deep PLN witness.

Phase 3: Demand-driven visitor path

Visitor as primary seam, explicit frame/worklist. Deep bio gate, PLN witnesses, then milestone ladder.

Phase 4: Trail/watermark execution spine

Reusable `SearchContext` threading. Full convergence to one canonical evaluator.

Red flags. Any binding mutation without exact rollback proof. Any speedup that breaks FC/bio witnesses. Letting medium benchmarks alone steer architectural choices. Mid-phase constant-factor measurement churn.

7 Planned: MORK and PathMap

From the MorkSpace boundary review (April 3):

- **mork:** is now a distinct public lane with explicit boundary. Generic (`new-space mork`) is disabled; use `mork:open-act!`.
- **Open:** `mork:close-space!` handle pool lifecycle.
- **Open:** Write-side public surgery semantics.

- **Open:** PathMap bridge ABI v2 (query-side-only bindings, varid dictionary, binary value encoding). Spec draft in `specs/pathmap_bridge_varid_abi.json`.
- MORK/PathMap should share semantics with the native backend — “a premium backend, not a second language” (architecture primer).

8 Lean Formalization Frontier

8.1 Completed (sorry-free)

- `EvalSpec.lean`: inductive operational semantics (6 mutual relations).
- `Eval.lean` + `Correctness.lean`: computable evaluator + soundness (`evalAtom_sound`, `interpretArgs_sound`).
- `TypedApplyMachine.lean`: abstract machine with 7 invariants.
- `TypedApplyDirect.lean`: direct recursive evaluator with `mergeEnv/extendBinder`.
- `TypedApplySoundness.lean`: `MachineArgChain` $\rightarrow$ `InterpretArgs` bridge.
- `BindingConsistency.lean`: loop-freedom preservation.
- `OwnershipInvariant.lean`: 3-way target safety.
- `ArgFrameMachine.lean`: argument-processing frame machine.
- `HE` $\leftrightarrow$ `PETTa` translation correspondence (5,400+ lines).

8.2 Needed

1. **IsValue/IsReducible formalization.** Precise definition of which atoms are values vs. reducible. Must rule out “no equations = value” as unsound. Positive examples (symbols, grounded values) and negative examples (arithmetic expressions, nested `if`).
2. **Direct $\approx$ frame-based equivalence.** Prove that direct recursive typed application and frame-based typed application agree on observable outcomes. Supports the C refactor in §5.
3. **Visible-binding projection correctness.** Formalize the branch-delta transport discipline: rewriting bodies with visible bindings and carrying only the local delta forward.
4. **Tabling correctness.** Connect `TableStore` semantics to `EvalSpec`: cached answers are sound (subset of complete evaluation).

9 Specification Artifacts

Seven JSON specification files in `specs/`:

File	Contents
<code>runtime_callables.json</code>	100+ callable registry with arity, effect class, profile membership, oracle status.
<code>runtime_effect_classes.json</code>	13 effect classes with memo and scheduler policies.
<code>he_runtime_3layer_matrix.json</code>	25/25 kernel, 69/69 grounded, 70/70 stdlib implemented.
<code>runtime_surface_audit.json</code>	Positive/negative test coverage for Minimal-MeTTa surfaces.
<code>he_backend_choice_audit.json</code>	Compatibility policy for HE-facing surfaces.
<code>pathmap_bridge_varid_abi.json</code>	Draft ABI v2 for the CeTTa↔MORK bridge.
<code>he_runtime_impl_matrix.json</code>	Cross-layer implementation status matrix.

10 Capability Ledger

A capability ledger tracks every nontrivial evaluator guarantee as a named contract with witnesses and a code anchor. Every serious branch update must preserve the entire ledger. This prevents the failure mode where “the model optimizes for the last thing it touched” (GPT-5.4 Pro).

Guarantee	Semantic witness	Perf witness	Code anchor
<code>let</code> single-result TCO	<code>countdown-let</code> 5000 → done	wall time, no stack overflow	<code>metta_call</code> let fast path, TAIL_REENTER
<code>chain</code> single-result TCO	<code>countdown-chain</code> 5000 → done	wall time, no stack overflow	<code>metta_call</code> chain fast path, TAIL_REENTER
<code>let*</code> desugaring TCO	nested <code>let*</code> → <code>let</code> chain	tail special forms test	<code>metta_call</code> <code>let*</code> handler
<code>if</code> single-result TCO	<code>countdown</code> 100k → done	< 2s on main	<code>metta_call</code> if handler, TAIL_REENTER
<code>select/once</code> stop	<code>test_select_match_stop</code>	terminates	select consumer stop propagation
Match transport order	<code>test_match_transport_order</code>	transport order	match source de- livery
<code>let*</code> visible env	<code>test_let_star_visible_env</code>	let env manipulations	body-visible pro- jection
Equation dispatch	<code>he_b1_equal_chain</code>	correct chain- ing	<code>query_equations</code>
Typed application	<code>he_d5_auto_types</code>	correct	<code>metta_eval_bind_typed</code>
Grounded ops	<code>he_c1_grounded_basic</code>	correct	grounded dis- patch

Guarantee	Semantic witness	Perf witness	Code anchor
Build cleanliness	<code>make BUILD=core</code>	zero warnings	Makefile + all headers
Effect classification	compile-time	—	<code>cetta_effect_policy.h</code>
Hashcons sharing	<code>test_borrowed_hashcons</code>	<code>insert count > 0</code>	HashConsTable on persistent arena
Full test suite	<code>make test</code>	$\geq 340/0$ < 5 min	Makefile test target

Usage. Before any branch update that touches the evaluator core, generate a report covering:

- build status,
- canary test matrix (every semantic witness above),
- benchmark mini-matrix (every perf witness above),
- changed files touching evaluator core,
- which optimization flags are on.

If any row in the ledger regresses, the update does not land until the regression is fixed or explicitly approved.

11 Principles

Extracted from review bundle guidance and development experience:

1. **No shortcuts for speed.** The “no equations = constructor value” hack broke arithmetic and nested evaluation. Speed improvements that violate semantics always regress later.
2. **Semantic fence first.** Before optimizing, run the witness set (medium benchmarks, deep benchmarks, structural canaries). Never let one benchmark steer architecture.
3. **Ablate optimizations.** Never bundle 6 features. Port and verify them one at a time so breakage is isolated.
4. **Lean as oracle.** Formalize the right invariants first, then implement in C. Not the other way around.
5. **Phase discipline.** Don’t optimize until the previous phase terminates correctly. Measurement mid-phase is churn.
6. **Keep default healthy.** Experimental gates (`CETTA_EXPERIMENT_*`) let two paths coexist safely during refactoring.

7. **Preserve what works.** Do not delete working features (`TAIL_REENTER`, `cetta_effect_policy.h`, `metta_eval_bind_typed`) to “make room” for new code. Add, don’t replace, until the replacement is proven correct.
8. **Capability ledger, not just a roadmap.** Track guarantees you already had, not just features you hope to add. Every guarantee needs a semantic witness, a performance witness, and a code anchor. The ledger is mandatory for every evaluator-touching branch update (§10). “The model will optimize for the last thing it touched unless the branch itself tells the truth” (GPT-5.4 Pro).
9. **Regenerate, don’t hand-carry.** Benchmark claims and bug root-cause stories must be regenerated from the actual current source, not inherited from prior sessions. If the code changed, the diagnosis may have changed with it. The core-opt triage demonstrated this: the query-cache $O(n^2)$ diagnosis was correct for an earlier state but stale for the current code.
10. **Hybrid fast-path / frame-machine design.** Use `TAIL_REENTER` for deterministic single-result fast paths (`let`, `chain`, `if`, `case`). Use `NativeResumeStack` only for genuinely deferred, multi-result, or owned-state paths (typed application, collapse, fold, select). Never replace the cheap path with the general mechanism.